

Game Design Documentation

Monster Dungeon

Version 1.2



Contents

1	Game Core	1
1.1	The Most Significant Design in the Game	1
1.2	The Core Design Applied in My Game	1
2	Game Program Structure	2
2.1	Message Bus + Game Message & Field Distribution	2
2.2	Ports	2
2.3	Game State	2
2.4	Player	2
2.5	Enemy	2
2.6	Item	2
2.7	Inventory	2
2.8	World	2
3	Player	3
3.1	Module Description	3
3.2	System Pipeline	3
3.3	SubModule Pipeline	3
3.3.1	Player Attack	3
3.3.2	Player Health	3
3.3.3	Player Movement	3
3.3.4	Player Stamina	3
4	Fog of War	4
4.1	How to Implement Fog of War?	4
4.2	How to Optimize the Fog of War?	5
5	Random Dungeon Map	6
5.1	Dungeon Map Randomization Principles	6
5.2	How to Generate a Proper Dungeon Map?	6
5.3	Enemy/Enemy Difficulty Randomize	9
5.4	Treasure Randomize	9
6	Enemy	10
6.1	Module Description	10
6.2	System Pipeline	10
6.3	SubModule Pipeline	10
6.3.1	Enemy Health	10
6.3.2	Enemy Attack	11
6.3.3	Enemy Movement	11
7	Version	13
7.1	Version 1.0	13
7.2	Version 1.1	13
7.3	Version 1.2	13
A	Hierarchy	14
B	Player	15
B.1	[1] Player Main Config parameters explanation.	15
B.2	[2] Player Juice parameters explanation (Play Feelings).	15
B.3	[3] Player Messages	15
C	Enemy	17
C.1	EnemyMovement Direction Submodule	17
C.2	PathFinding Algorithm	17

1 Game Core

This is a dungeon level-up game. The player collects what they need to clear each level, and the final goal is to defeat the Boss.

1.1 The Most Significant Design in the Game

Before reading this chapter, let's think about what makes a game fun and keeps players playing. A fun game comes from these things:

1. The Core: Decision.
2. The heartbeat around the core: Uncertainty.
3. The decoration of the decision: the density of decisions and their consequences.

A fun game is packed with interesting decisions. The character, eye-catching visuals, and UI are the decoration around those decisions.

The decisions serve the core loop: beat enemies, feel your growth, and take on challenges that never seem to end.

1.2 The Core Design Applied in My Game

The game has several systems: Player, Fog of War, Task, Random Dungeon, Enemy, and more.

First, Decision: the core loop is that the player collects equipment, enters the dungeon, defeats enemies, reaches the exit portal, and chooses to extract or enter the next, harder level. The player's final goal is to defeat the Boss.

Second, Uncertainty: to create uncertainty, I added a Fog of War system and used an algorithm to generate the random dungeon map.

Third, the decoration: to make decisions more interesting, I added the Task System, wrote some story, and designed various weapons, monsters, and chest loot, giving players more choices. I also optimized the dungeon generation algorithm to make maps more balanced and create more decision points in combat.

2 Game Program Structure

The game uses a message bus + Game Message to decouple its systems. Each system manages its own scripts. One system handles one object type, and one script handles one function. Different systems connect through the message bus.

Before this structure, I tried a Game Manager / Director pattern to control the whole game system, but it lost the battle: different functions became coupled together. Then I combined the Game Manager with a state machine and a message bus to organize the code. But this defect surfaced as the code grew past about 13k lines. The Game Manager and the state machine could be separated into one independent system that publishes the game state. At September 30, 2026, I completed the new code structure.

Designer Note:

2.1 Message Bus + Game Message & Field Distribution

The Message Bus + Game Message base class system send messages and completed data for different systems or fields, different fields owns different message bus and The Supreme Message Bus manages messages among different fields.

Distribute functions into these fields: Game State, Player, Enemy, Item, Inventory, World and independent function module names 'Execution & Static Scripts'.

2.2 Ports

1. Directly Quote: In modules, deal with massive data with high speed.
2. Message Bus: Event Communication that publish significant events.
3. Shared Data: modules one another with massive data communications.

2.3 Game State

1. Communication: Only Message Bus.
2. Responsibility: Declare the Game State
3. State types: Main World, Dungeon + Layer.
- 4.

2.4 Player

2.5 Enemy

2.6 Item

2.7 Inventory

2.8 World

3 Player

3.1 Module Description

1. Player Config: Store the player's initialization config file.
2. Player Attack: Properties: Control the player attack mode. Describe at [3.3.1](#).
3. Player Health: Properties: Control the player health. Describe at [3.3.2](#).
4. Player Movement: Properties: Control the player movement. Describe at [3.3.3](#).
5. Player Stamina: Properties: Control the player stamina. Describe at [3.3.4](#).

3.2 System Pipeline

1. Keyboard input detection: Control the player's movement and summon the functions.
2. Properties: Player Renderer, Player Attack, Player Movement, Player Stamina.

3.3 SubModule Pipeline

3.3.1 Player Attack

3.3.2 Player Health

Core Data: currentHealth.

1. PlayerHealthInit.cs: Initial the player health and transmit the player health into the PlayerHealthCache.cs.
2. PlayerHealthCache.cs: Maintain the **currentHealth** argument.
3. PlayerHealthCalculator: Control the currentHealth changing port.
4. MaxHealthManager.cs: manages the maximum/minimum health port.
5. Invincibility: When invincibility, player will not take damages from anywhere.
6. HitReactions: Control the effects.
7. PlayerHealthMessage: Send messages to Plug HealthBar folder to control the currentHealth display.

3.3.3 Player Movement

3.3.4 Player Stamina

4 Fog of War

As described above, the Fog of War increases uncertainty.

4.1 How to Implement Fog of War?

Use League of Legends (LOL) as the main reference. The simplest way to achieve this is a shader; the Built-in rendering pipeline is enough.

The logic of the Fog of War system (design decisions made by myself):

1. Cast rays: 30 rays from the camera (started with 360, but that was unnecessary and costly). If a ray hits no obstacle, it reaches the view radius of 10; the end points connect into a fan shape.
2. Render the shape into a small image: a fog camera renders the visible and invisible areas into a 256×256 texture.
3. Make the fog feel like LOL: apply Gaussian blur to the small image, then use the Interpolate shader to blend between the old and new shapes smoothly. I learned this step from Bilibili (BV1Wr421K7Zf, 5:40).
4. Hide enemies but not the scene: the Fog Overlay shader samples the scene. Where the fog is dim (alpha = 0.8), the scene is hidden; where it is clear, the scene stays visible.
5. Extend the visible area 0.6 units beyond each obstacle, so the obstacle itself stays visible. This improves readability.

Before this, I tried a transitional zone, but it looked bad. I tried a multi-layer transitional zone too, but it was still bad. Finally, I found the Gaussian blur method on Bilibili and applied it to the project — it worked!

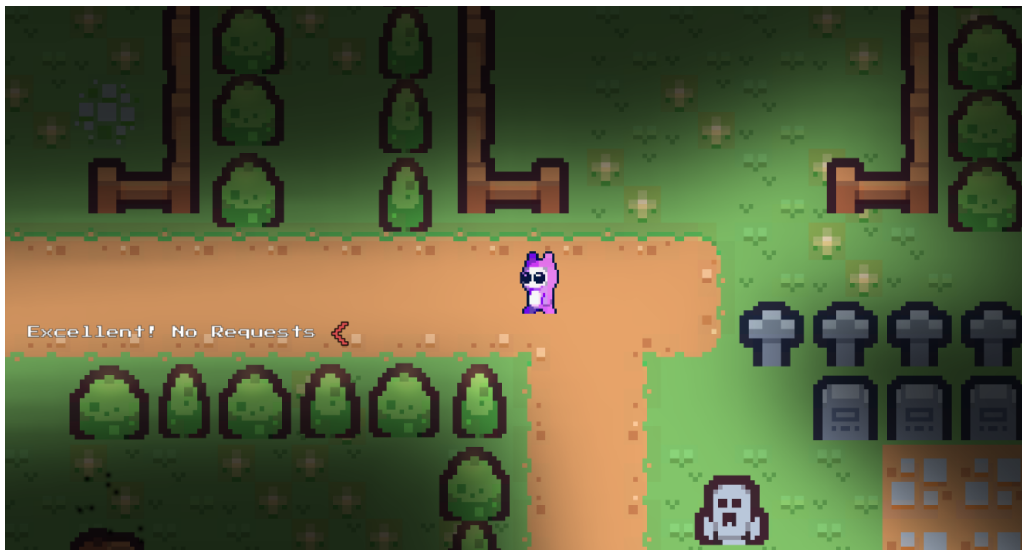


Figure 1: Fog of War effect

4.2 How to Optimize the Fog of War?

1. Cut down the rays. 360 or 180 rays are unnecessary and costly. I tried 15–45 rays — that was enough, and the experience improved a lot. Below that, uncertainty drops; above that, the cost becomes too heavy for other computers.
2. Detect the player's movement: when the player stops moving, skip recomputing the fog. I tried computing the shape only once in a while (every 0.2 s) to save CPU, but it failed because the shape could not follow the camera center.

Designer's Note: The fog is a complete system — not because it's hard to render, but because it turns every step into a choice. I cut the rays from 360 to 30; the players never notice the difference, but the CPU does.

5 Random Dungeon Map

The random dungeon map is the biggest source of uncertainty and the reason players keep coming back to the same dungeon level.

5.1 Dungeon Map Randomization Principles

A random dungeon map should follow these principles:

1. The road between the exit portal and the spawn point must be accessible.
2. The terrain must be playable: it needs walls, plenty of open space for walking and fighting, and random obstacles that force players to use strategy.

5.2 How to Generate a Proper Dungeon Map?

To generate a proper map, I recommend the steps below — I have tried them, and they work (The new method is complete and working). This is the design I am most proud of, because it directly supports my core gameplay loop.

New Method:

1. Use the cellular automaton algorithm to dig the terrain.
2. Keep the biggest open area generated by the algorithm. (This is the core optimization step.)
3. Place the Spawn Point and the Exit Portal randomly inside the open area. Put the spawn point in an open spot, and put the exit as far away as possible, also in an open spot.
4. Scatter obstacles around the map.
5. Check that the road between the exit portal and the spawn point stays accessible. If obstacles block the main road, scatter fewer obstacles.
6. Make sure the Spawn Point and the Exit Portal sit on ground tiles; if not, re-run step 3.
7. Tile the map and place the prefabs.

Here Shows the effects of the algorithm.

Old Method:

1. Initialize the elements. Set the map scale first: the map is a square with a fixed border. Then place a spawn point and an exit portal in the right spots. In my design, the distance between them should be at least 80% of the border length, and the exit portal should sit within 95% of the border length so it never spawns on the border itself.
2. Guarantee that the road between the exit portal and the spawn point is accessible. The A* algorithm can guarantee a spine road, but it needs optimization — if we generate the spine road with A* first, the road has no randomization.

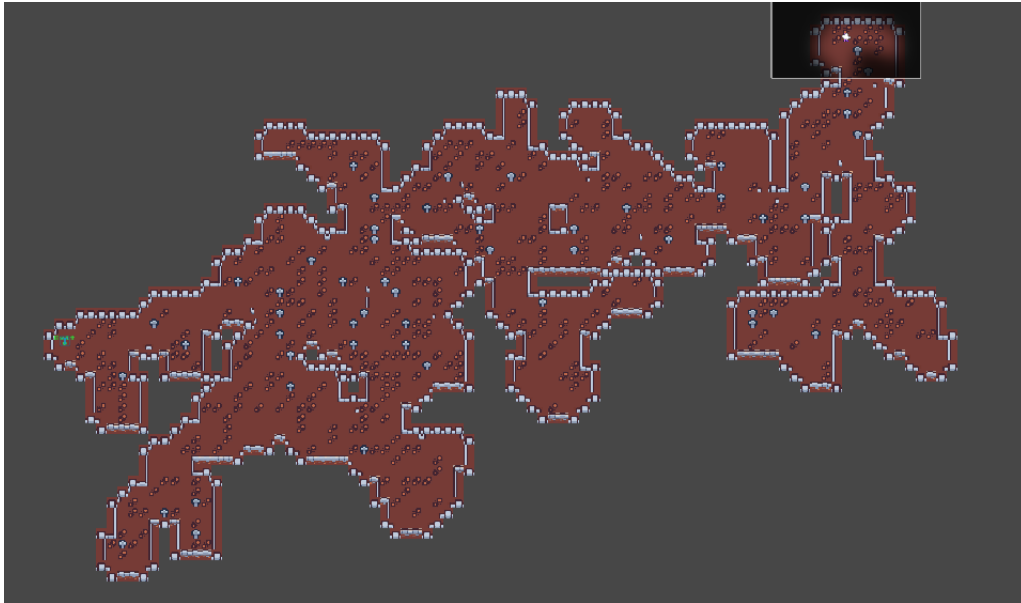


Figure 2: New dungeon generation method

3. Cellular automaton is a good fit for terrain generation. Use it at this step to generate random terrain. Its flaw: it tends to create karst-like terrain with stalactites, and the stalactites make wall generation awkward — walls look strange when a stalactite has a pointed top.
4. Cellular automaton may cover the spine road and break the path between the spawn point and the exit portal. This step needs optimization — it becomes unnecessary when the earlier steps are handled properly.
5. Once the spine road is generated, use BFS to inject more uncertainty into the map, giving players more adventurous decisions. Because of BFS's nature, it creates many side roads unrelated to the spine road, which can confuse the player.
6. Dig rooms for the spawn point, the exit portal, combat, and chests. But rooms shouldn't sit right beside the spawn or exit points — that would reduce the adventure uncertainty.
7. Scatter random obstacles along the accessible roads. This step has no trick to it, but don't block the essential path with obstacles.
8. Run an accessibility check — the road must stay accessible. But this step is redundant if checking is already done in steps 2 and 4. To optimize: check the road once, and prepare a fallback plan in case the road is blocked.
9. To make wall generation more reliable, cut off the stalactite's pointed top — otherwise the walls cannot connect properly.
10. Build the topology of the dungeon map and tile the map. Define each map element as an inside dot, a border dot, or an outside dot. This helps scripts understand tile locations: inside dots render floor, border dots render walls, outside dots render nothing.

11. Create the exit portal prefab and the enemy spawn area. The enemy spawn area is described in section 5.3.
12. Send messages to the state machine and the player. When the state machine receives the message, it changes its status to 'dungeon'; the player is placed at the spawn point.
13. Add Collider2D to walls and obstacles.

Tips: the steps above have several defects, so I will reorder them to optimize the pipeline — using fewer compute resources and generating a more playable dungeon map. The picture below shows version 1.0 of the random dungeon map.

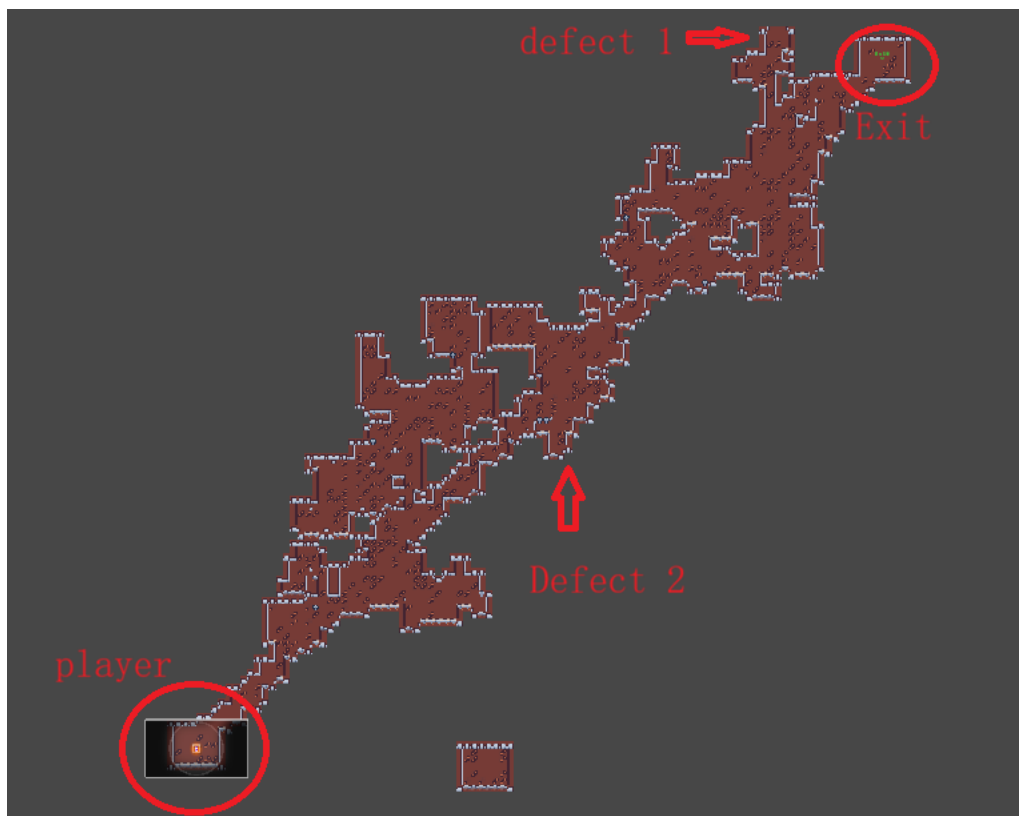


Figure 3: V1.0 dungeon defects

Description of the picture elements:

1. Player: the player's spawn point.
2. Exit: the exit portal's location
3. Defect 1: the walls don't connect properly, so the player can walk out of the dungeon.
4. Defect 2: walls appear in the wrong places, making the map look strange.
5. Defect 3: the exit's location is fixed, so the map doesn't show its randomness.
6. Defect 4: the open areas are too small, so the player can't fight comfortably in the map.

5.3 Enemy/Enemy Difficulty Randomize

5.4 Treasure Randomize

6 Enemy

6.1 Module Description

The system is divided into the following modules.

- **Spawn Enemy:** The only entry point for other modules to send spawn messages.
- **Public Config:** Stores the enemy's static template data.
- **Public Script:** Stores the monster's public behaviour (the Core Pipeline).
- **Special Enemy AI:** Stores the special enemy AI scripts. These scripts modify the core pipeline to change behaviours.
- **EnemyMessageBus:** Provides a private message bus for the enemy system's internal use.

6.2 System Pipeline

Step 1: A module sends a spawn message to the spawn module. The message contains: monster species, count, and locations.

Step 2: The Enemy Spawn Module subscribes to spawn messages; `EnemyPipelineController.cs` begins to control the pipeline.

Step 3: Initialize the matching template data.

Step 4: Create a Prefab with the correct Unity components: `SpriteRenderer`, `CircleCollider2D/3D`, `Rigidbody2D/3D`.

Step 5: Public scripts attach to the Prefab and control the monster's behaviour (the Core Pipeline, described in [Section 6.3](#)).

Step 6: If the monster carries a special function, the matching Special Enemy AI script modifies the core pipeline to change its properties.

Step 7: Add the sprites and animations, then transmit the Prefab to the Enemy Spawn Module.

Step 8: The Enemy Spawn Module sends the Prefab to the renderer.

6.3 SubModule Pipeline

6.3.1 Enemy Health

Core Data (`currentHealth`):

1. `\Assets\EnemyUnit\PublicScript\EnemyInit.cs`: Data Initialization Region. Copies the template data into `EnemyHealthData.cs`, which maintains the "`currentHealth`" argument.
2. `\Assets\EnemyUnit\PublicScript\EnemyHealthData.cs`: Maintains the "`currentHealth`" data.

3. `\Assets\EnemyUnit\PublicScript\Direction.cs`: Receives and calculates damage. The only entry point that accesses “currentHealth” in `EnemyHealthData.cs`. The submodule’s introduction is placed in Appendix C.1.
4. `\Assets\EnemyUnit\PublicScript\EnemyDeath`: Destroys the `EnemyPrefab` and plays the death animation.

Plugs (Relative Path):

1. `\..\Buff\EnemyBuffSystem.cs`: The “DamageBuff” — only handles debuff damage dealt by the player.
2. `\..\HealthBar`: The enemy’s health bar controller.
3. `\..\Effects`: Controls various effects, such as hit effects and particle effects.
4. `\..\Protects`: Guards against unexpected runtime incidents, such as the renderer getting stuck or enemy prefabs getting stuck in walls.
5. `\..\DamagePopUp`: Damage numbers that pop up when the player deals damage to the enemy.

Resources:

1. “HitParticlePrefab”, controlled by `HitParticle (\..\EnemyHealth\Effects\HitParticle)`.
2. “EnemyPrefab”, located at `\Assets\Resources\Enemy\EnemyPrefab`.
3. “DamageNumberPrefab”, controlled by the `DamagePopUp` plug (`\..\EnemyHealth\DamagePopUp`) (instantiated by scripts).

6.3.2 Enemy Attack

Attacks are triggered only by Collider2D. When the enemy’s `Collider2D` touches the player’s collider, the enemy sends attack data to the player’s “PlayerHealth” module.

6.3.3 Enemy Movement

Relative Path: `\Assets\EnemyUnit\PublicScript\EnemyMovement`

Core Data:

1. `\..\EnemyMovementInit.cs`: Initializes the movement data into `EnemyMovementData.cs`.
2. `\..\EnemyMovementData.cs`: Maintains the movement data — “current direction” and “current speed”.
3. `\..\EnemySpeedCalculator.cs`: The only port that controls the current speed. Calculates the final speed and transmits the `CurrentSpeed` data into `EnemyMovementData.cs`.
4. `\..\Direction`: The only port that controls the current direction. Calculates the final direction and transmits the `CurrentDirection` data into `EnemyMovementData.cs`.

Plugs:

1. `\..\EnemyKnockBackApplier`: Controls the enemy's knockback.
2. `\..\PathFindingAlgorithm`: Controls the enemy's direction, guiding enemies along the correct path to catch the player.

7 Version

7.1 Version 1.0

Published the primary demo but unfit for playing because the dragging function and sell items function not be inserted. The game code structure, optimizing are not completed and the task, game story has not set. Updated at September 24, 2026.

7.2 Version 1.1

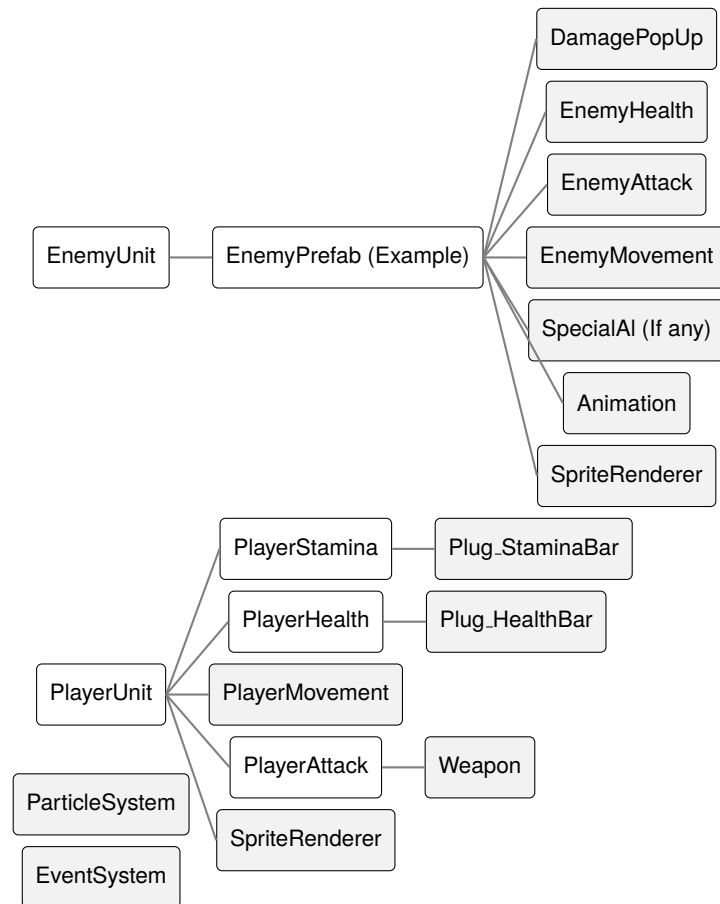
Reconstruct the code structure: Applied the new structure to consolidate the structure so as to add new function (Dragging Function & Sell Items Function) in the game. Optimized the code structure and system pipelines. Completed the Game Design Documentation. Updated at September 30, 2026.

7.3 Version 1.2

Added the Dragging Function & Sell Items Function in the game.

Appendix

A Hierarchy



B Player

B.1 [1] Player Main Config parameters explanation.

Path: \Assets\PlayerUnit\PlayerConfig\PlayerConfig.cs

SubModule: PlayerHealth

Parameters	Function	Data First Read
MaxHealth	skim	PlayerHealth.cs: cs.44
Defense	skim	PlayerHealth.cs: cs.45
InvincibleDuration	Invincible time	PlayerHealth.cs: cs.152

Table 1: Player main config parameters

Plug (PlayerHealth): PlayerHealthBar

Path: \Assets\PlayerUnit\PlayerHealth\Plug_HealthBar

Files Introduction:

- HealthBarConfig: Control the HealthBar's Animation.
- HealthBarUI.DataTransmittion: Read the data.
- HealthBarUI.Assembly: Set the UI.
- HealthBarUI.Animation: Record the animation types.
- HealthBarUI.Display: Display the HealthBar's Animation.

B.2 [2] Player Juice parameters explanation (Play Feelings).

Path: \Assets\PlayerUnit\PlayerConfig\PlayerConfig.cs

B.3 [3] Player Messages

Path: \Assets\PlayerUnit\PlayerMessageBus

Messages SubClass:

1. PlayerHealth

Path: \Assets\PlayerUnit\PlayerHealth\PlayerHealthMessage.cs

Duty: Transmit the CurrentHealth and MaxHealth from PlayHealthCache to Plug_HealthBar, in order to display the "CurrentHealth\ MaxHealth" text.

Sender: \Assets\PlayerUnit\PlayerHealth\PlayerHealthCache.cs: cs.53-57.

Receiver: Assets\PlayerUnit\PlayerHealth\Plug_HealthBar\HealthBarUI.DataTransmission:cs.36.

2. PlayerAttack

Path: \Assets\PlayerUnit\PlayerAttack\PlayerAttackRequestMessage.cs

Duty: Transmit three arguments: Direction, PlayerPosition, MousePosition to WeaponSettingsModule.

Sender: \Assets\PlayerUnit\PlayerAttack\PlayerAttackAction.cs: cs.65-70.

Receiver: \Assets\PlayerUnit\PlayerAttack\WeaponSettings\Iweapon.cs: cs.28.

C Enemy

C.1 EnemyMovement Direction Submodule

Relative Path: \Assets\EnemyUnit\PublicScript\EnemyMovement\Direction

1. Direction Data Maintainance: EnemyDirectorCalculator.cs.
2. Avoid the obstacle: EnemyAvoidance.cs.
3. Seperate the enemies movement(do not let enemies engaged together): EnemySeperation.cs.
4. Decide the state whether to “Patrol” or “Chase”.
5. Use the BFS algorithm to optimize the chasing algorithm. The algorithm placed in the “PathFingingAlgorithm” folder and the algorithm’s description placed in [C.2](#).

C.2 PathFinding Algorithm

The BFS(Broad Field Searching) algorithm will increase the enemy chasing logic, preventing enemy chasing like a noob. The BFS algorithm achieved by following steps:

1. Scan the whole map into states that combined with 'can walk', 'cannot walk'.
2. Applied the BFS algorithm into the EnemyMovement’s Direction data flow to adjust the moving direction so that enemy will follow the correct path to chase the player.
3. **The BFS algorithm’s arguments have written in the BFS_Arguments.cs.**